

DevSecOps PaaS — Guide de démonstration (Soutenance)

Projet: Plateforme PaaS DevSecOps avec microservices

VM lab: master@192.168.56.129

Date: Juillet 2026

Connexion à la VM

```
ssh master@192.168.56.129
cd ~/devsecops_paas_microservices
```

0. Avant la démo — Vérifier que tout fonctionne

```
bash paas/scripts/lab.sh health
bash paas/scripts/lib/check-paas-lab-health.sh
```

Si la VM vient de redémarrer :

```
bash paas/scripts/lab.sh start
# Attendre 5-15 minutes, puis relancer health
```

1. VM et cluster Kubernetes

```
hostname
uname -a
free -h
df -h /

kubectl get nodes -o wide
kubectl version --short
```

À dire au jury : Cluster k3s sur VirtualBox — 1 master + 2 workers, Kubernetes léger pour valider le PaaS.

2. Structure du projet

```
cd ~/devsecops_paas_microservices
git log -1 --oneline
tree -L 2 paas/ 2>/dev/null || find paas -maxdepth 2 -type d | head -30
```

Dossier	Rôle
`paas/frontend/`	Interface PaaS + API (Next.js)
`paas/jenkins/`	Pipeline 12 étapes (`Jenkinsfile.paas-deploy`)
`paas/gitops/`	Chart Helm pour les applications utilisateur
`paas/k8s-manifests/`	Postgres, Kyverno, manifests hébergés
`paas/scripts/lab.sh`	Opérateur lab (start, health, deploy, heal)

3. Services plateforme (namespace `paas`)

```
kubectl get all -n paas
kubectl get pods,svc,ingress -n paas -o wide
kubectl get pvc -n paas

curl -sS http://192.168.56.129:30100/api/health | python3 -m json.tool
```

Navigateur : <http://192.168.56.129:30100/login>

4. Stack DevSecOps — Tous les namespaces

```
kubectl get ns
kubectl get pods -A | grep -E 'paas|cicd|harbor|sonarqube|dependency|argocd|kube-system|traefik'
```

Par service

```
# Jenkins (CI/CD)
kubectl get pods,svc -n cicd

# Harbor (registry)
kubectl get pods,svc -n harbor

# SonarQube (SAST)
kubectl get pods,svc -n sonarqube

# Dependency-Track (SCA / SBOM)
kubectl get pods,svc -n dependency-track

# Argo CD (GitOps)
kubectl get pods,svc -n argocd
kubectl get applications -A

# Traefik (ingress des apps utilisateur)
kubectl get svc -n kube-system traefik -o wide
```

Tableau des ports

Service	Port	URL
PaaS UI	30100	http://192.168.56.129:30100/login
Jenkins	30090	http://192.168.56.129:30090
Harbor	30002	http://192.168.56.129:30002
SonarQube	30900	http://192.168.56.129:30900
Dependency-Track	30353	http://192.168.56.129:30353
Traefik (apps)	30659	http://{projet}.192.168.56.129.nip.io:30659/

5. Applications déployées (pods, services, ingress)

```
kubectl get ns --no-headers | awk '{print $1}' | grep -vE '^(kube-|paas|cicd|harbor|sonarqube|dependency-track|argocd|)'
```

Exemple pour un projet (warda-youssef) :

```
PROJECT=warda-youssef
kubectl get deploy,pods,svc,ingress -n "$PROJECT" -o wide
kubectl describe ingress -n "$PROJECT" | tail -20
curl -sS -o /dev/null -w "HTTP %{http_code}\n" "http://{PROJECT}.192.168.56.129.nip.io:30659/"
```

Toutes les routes ingress :

```
kubect1 get ingress -A
```

6. Pipeline Jenkins — 12 étapes

Navigateur : Jenkins → job paas-deploy → dernier build réussi.

Terminal :

```
kubect1 get pods -n cicd -l app=jenkins
head -80 paas/jenkins/Jenkinsfile.paas-deploy
```

Les 12 étapes

#	Étape	Outils	Blocage
1	Validation params	Groovy	Requis
2	Git checkout	Git	Requis
3	Build application	Maven, npm, pip	Requis
4	SCA	Dependency-Check, CycloneDX,	Obligatoire
5	SAST	SonarQube	Obligatoire
6	Image Docker	Docker, crane → Harbor	Requis
7	Helm chart	Helm	Optionnel
8	Artifactory	—	Optionnel
9	Cosign	Signature d'image	Obligatoire
10	ZAP	OWASP ZAP	Optionnel
11	Helm OCI	—	Optionnel
12	GitOps	Commit values.yaml	Requis

Étapes obligatoires (échec = pas de déploiement) : 4, 5, 9 + gate sécurité plateforme.

7. GitOps et Argo CD

```
ls -la ~/gitops/apps/
cat ~/gitops/apps/warda-youssef/values.yaml | head -30

kubect1 get applications -n argocd
```

Flux : Jenkins build + signe → PaaS commit le tag dans GitOps → Argo CD sync → pod en cluster.

8. Sécurité — Kyverno et images signées

```
kubect1 get clusterpolicy 2>/dev/null | head -10

kubect1 get deploy -n warda-youssef -o jsonpath='{.items[0].spec.template.spec.containers[0].image}'
```

9. Déroulé recommandé (5–10 min)

Étape	Action
-------	--------

1	`kubectl get nodes -o wide` — montrer le cluster
2	Ouvrir PaaS UI — login, liste projets, vue pipeline
3	Ouvrir Jenkins — logs Step 4 (SCA) et Step 9 (Cosign)
4	`kubectl get deploy,pods,svc,ingress -n <projet>`
5	Ouvrir l'URL live de l'app
6	Montrer Harbor — image poussée
7	Montrer SonarQube / Dependency-Track
8	`cat ~/gitops/apps/<projet>/values.yaml` — tag promu

10. Script rapide « tout-en-un »

```

NODE=192.168.56.129
echo "=== HEALTH ===" && bash paas/scripts/lab.sh health
echo "=== NODES ===" && kubectl get nodes -o wide
echo "=== PAAS ===" && kubectl get pods,svc -n paas
echo "=== CICD STACK ===" && kubectl get pods -n cicd,harbor,sonarqube,dependency-track 2>/dev/null
echo "=== USER APPS ===" && kubectl get ingress -A
echo "=== URLS ==="
echo "  PaaS:      http://${NODE}:30100/login"
echo "  Jenkins:   http://${NODE}:30090"
echo "  Harbor:    http://${NODE}:30002"
echo "  Sonar:     http://${NODE}:30900"
echo "  App ex:    http://warda-youssef.${NODE}.nip.io:30659/"

```

11. Dépannage rapide

Problème	Commande
App 404 (Traefik)	`bash paas/scripts/lab.sh heal <projet> <build> 8080`
DB / Prisma	`bash paas/scripts/lab.sh db-repair`
VM après reboot	`bash paas/scripts/lab.sh start`
Routing Traefik	`bash paas/scripts/lib/lab-fix-traefik-app-routing.sh`
Rebuild frontend après fix	`bash paas/scripts/lab.sh frontend`

Conseils pour la soutenance

1. Commencer par le navigateur (PaaS UI) — plus visuel pour le jury.
2. Garder le terminal pour kubectl get nodes, namespace paas, et une app déployée.
3. Préparer 1–2 projets qui répondent HTTP 200 avant la présentation.
4. Slides complètes : docs/DEVSECOPS_PAAS_PRESENTATION_30_SLIDES.md

Document généré pour la soutenance DevSecOps PaaS — VM lab 192.168.56.129